

Script-Controlled ACL - Restrict Record Access Based on Field Value

ServiceNow Micro-Project

PHASE 1 | BRAINSTORMING & IDEATION PHASE

1. Brainstorming & Ideation Phase

1.1 Background

Organizations that run on ServiceNow frequently store sensitive, department-specific data in custom tables that are shared across the whole institution. A common example is an academic institution that keeps student and faculty records for multiple branches (ECE, EEE, CSE) inside a single table. Left unprotected, any authenticated user with basic read access to that table can view records belonging to every branch, which is a data-privacy and governance risk. The idea for this project originated from that exact problem: how do we let ServiceNow enforce "see only what belongs to you" at the record level, automatically, without relying on users to filter data themselves?

1.2 Problem Statement

Design and implement a mechanism inside ServiceNow that restricts visibility and modification rights on the **Institution Details** (`u_institution_details`) table so that:

- Only users who belong to the EEE branch can view, create, edit, or delete records where **Branch = EEE**.
- Administrators retain complete, unrestricted access to all records regardless of branch.
- Users who do not hold the required branch role are denied access entirely, with no accidental data leakage.

1.3 Ideas Explored

Option Considered	Description	Verdict
Field-level ACL only	Restrict access to individual fields (e.g., hide the Branch column) rather than whole records.	Rejected — does not stop users from opening records of another branch; only hides one field.
Business Rule based filtering	Use a "before query" Business Rule to append an encoded query filter for non-admin users.	Rejected as primary mechanism — Business Rules are good for query augmentation but do not natively enforce read/write/create/delete security the way ACLs do, and can be bypassed by direct record access (<code>sys_id</code> navigation).
Simple role-based ACL (no script)	Grant access purely by role, without checking the Branch field value.	Rejected — a role alone cannot differentiate an EEE record from a CSE record; it would grant access to all rows once the role is present.
Data Condition ACL (no script)	Use only the "Branch is EEE" condition, without a script.	Partially useful, but on its own cannot special-case administrators or combine multiple logic branches cleanly.
Script-controlled ACL (chosen)	Combine a Data Condition (Branch is EEE) with an Advanced/Script condition that explicitly allows admins, allows users with the branch-specific role, and denies everyone else.	Selected — most flexible, auditable, and secure; enforced by the platform at the record-security layer for every access path (form, list, Service Portal, API).

1.4 Why Script-Controlled ACL Was Chosen

- **Field-value awareness:** Only a script (or a data condition combined with a script) can compare the record's own *Branch* field against the requesting user's role at run time.
- **Granular operation control:** ServiceNow ACLs can be defined independently for *read*, *create*, *write*, and *delete*, which maps naturally onto a phased, role-by-role access model (`bb1` → `bb2` → `bb3` → `bb4`).
- **Centralized enforcement:** Once defined, the ACL is enforced everywhere — list views, forms, REST/Import API, and the Service Portal — so there is no code path that can bypass it.
- **Administrator override:** The script explicitly checks `gs.hasRole('admin')` first, guaranteeing administrators are never locked out even as branch-level restrictions tighten.

1.5 Initial Scope Ideas

In scope	Out of scope
• One custom table (Institution Details) • One test user (EEE User) and four custom roles (<code>bb1</code>–<code>bb4</code>) • Four ACL records (read, create, write, delete) driven by a script • Verification through user impersonation	• Multi-branch dynamic role mapping (e.g., auto-detecting branch from user profile) • Workflow/approval automation • UI Policies / Client Scripts for field-level styling • Integration with external identity providers

1.6 Ideation Outcome

The brainstorming session converged on a **script-controlled, role-and-field-driven ACL model** as the most secure and maintainable way to achieve branch-based record restriction in ServiceNow, while keeping the implementation small enough to be delivered as a focused 3-hour micro-project. This decision directly shaped the requirements, design, and development phases that follow.